Sprint 2 · Photo Input, Recognition and Confirmation

MealLens

A simple photo-based meal tracker

Ahmed Bahaj
Albaraa Alnahari
Yazeed Bafaqeeh
Ibrahim AbuAlghaith

CCSW 431
Mobile Programming
University of Jeddah · 2026

Sprint 2 goal

Deliver the meal-logging stories planned for Sprint 2

From a photo to a confirmed meal.

The user adds a photo, the app proposes the foods inside it, and the user corrects the items and portions before any estimate is shown.

Sprint scope

003 Photo input
004 Food recognition
005 Food confirmation

Early start 006 Nutrition estimation

What we built

Three working screens, running on iOS, Android and the web

Home

Greets the user, offers Scan a meal or Upload photo, and shows the most recent meal with a History tab.

Confirm your meal

Shows the detected dish with a confidence score. Every ingredient has an editable weight in grams and can be removed or added.

Meal summary

Total calories, a health score, protein / carbs / fat with percentages, sugar, fiber and sodium, and a per-ingredient breakdown.

Health flags

Clear warnings such as “Not suitable for diabetics”, high calories, high fat, sodium and allergens, plus positive notes like fiber and protein.

The Sprint 2 journey

Example: Ahmed logs a plate of masoub

01

Add a photo

Scan with the camera or upload from the gallery.

02

Recognise

The app proposes the dish and its ingredients.

03

Confirm

Adjust grams, remove or add ingredients.

04

Review

Read calories, macros and health flags.

05

Save

The meal appears in Recent meal and History.

Screen 1 · Home

The starting point for every meal

Two ways in

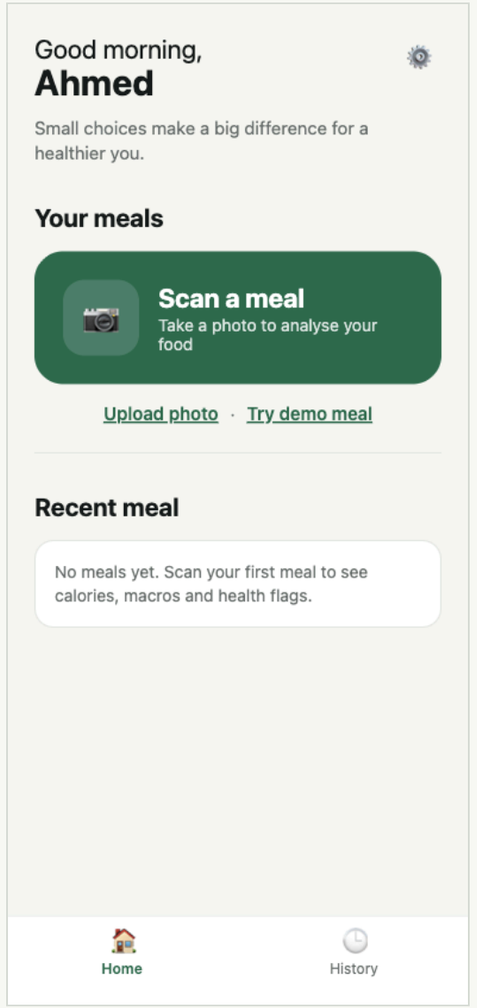
Scan a meal opens the camera. **Upload photo** opens the gallery. A demo meal link lets us test without a camera.

Recent meal

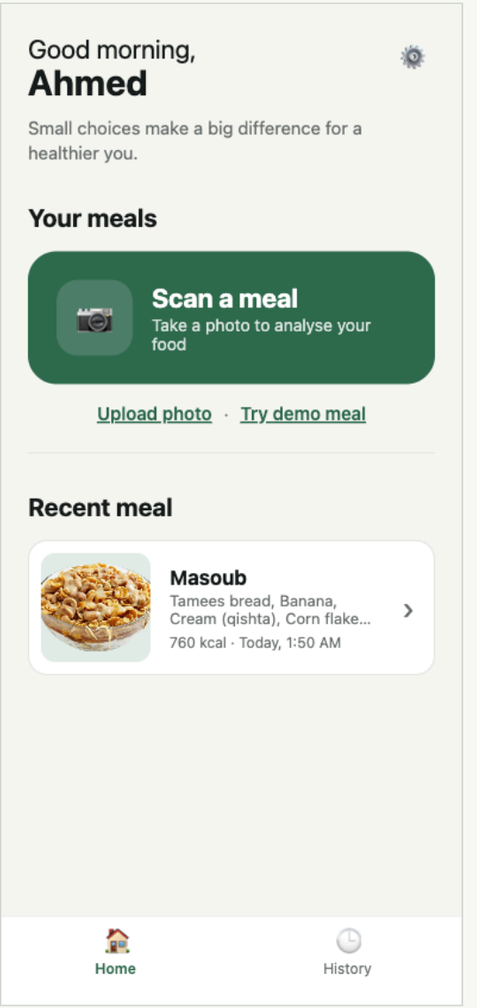
After saving, the latest meal appears with its ingredients, calories and time. Tapping it reopens the summary.

History

The History tab lists every saved meal of the session.



Before the first meal



After saving a meal

Screen 2 · Confirm your meal

The user stays in control of what was detected

Detected dish

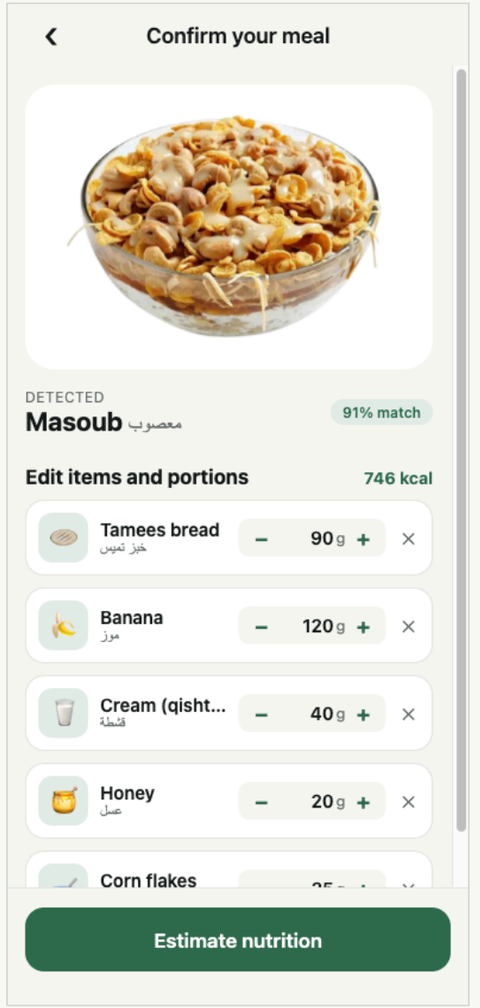
The photo is analysed and labelled **Masoub · معصوب** with a 91% match. Six ingredients are proposed with default portions.

Edit items and portions

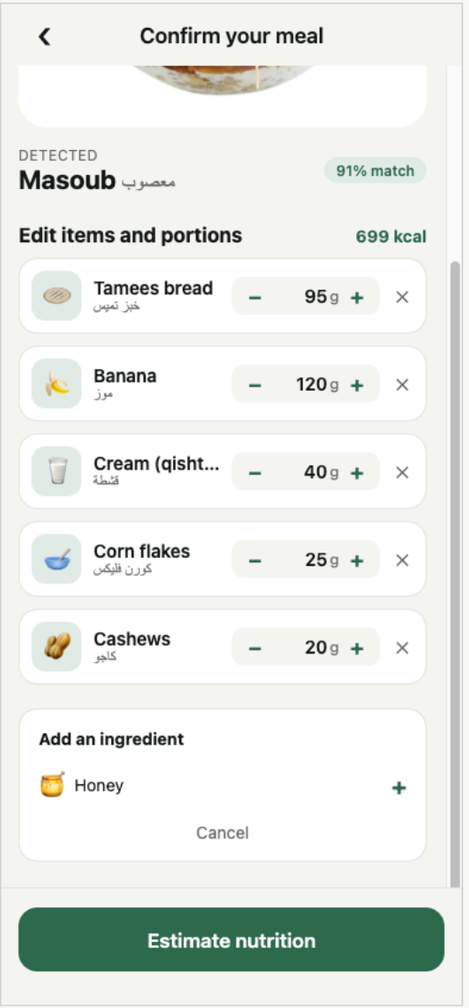
Each ingredient has – / + steppers and a gram field. Items can be removed with ✕ or added from the **Add missing ingredient** panel.

Live total

The calorie total updates instantly as portions change, before the user taps **Estimate nutrition**.



Detected ingredients



Editing and adding items

Screen 3 · Meal summary

Calories, macros, health flags and a per-ingredient breakdown

Estimate

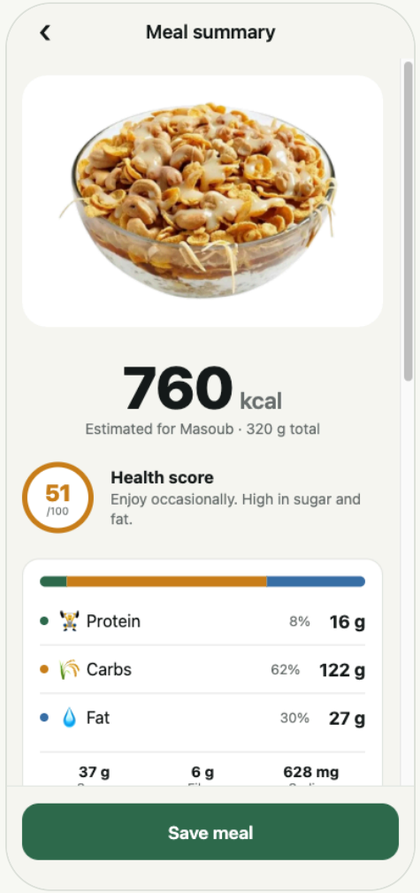
Total kcal, a health score out of 100, and a macro bar for protein, carbs and fat with sugar, fiber and sodium.

Health flags

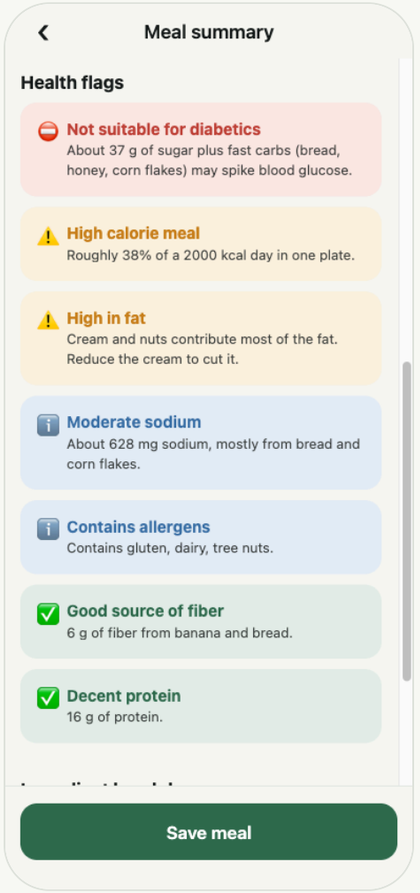
Rule-based flags warn about diabetes risk, calories, fat, sodium and allergens, and highlight positives.

Breakdown

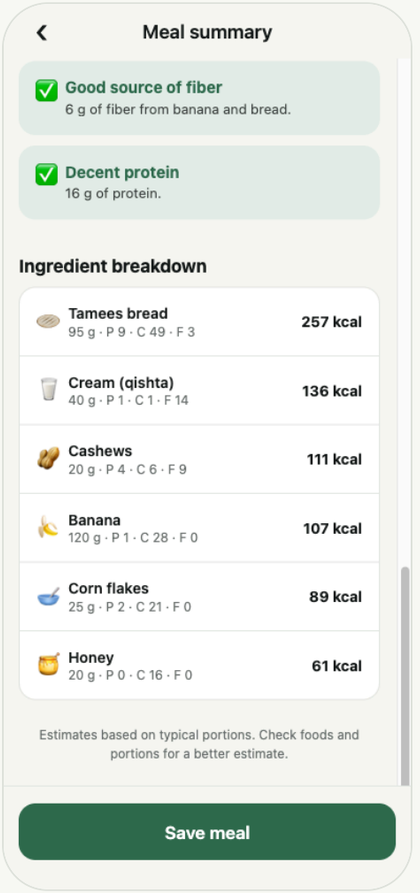
Every ingredient with its grams, macros and calories, sorted by impact. **Save meal** stores it.



Totals and macros



Health flags



Ingredient breakdown

How the estimate works

Recognition is mocked for now; everything after it is real and reusable

1 · Photo

Camera, gallery or demo image.

2 · Recognition

Returns a dish name, confidence and an ingredient list with default grams. Currently a mock that returns masoub.

3 · Confirmation

The user edits grams, removes or adds ingredients.

4 · Nutrition table

Per-100 g values (kcal, protein, carbs, fat, fiber, sugar, sodium) scaled by the confirmed grams and summed.

5 · Flags & score

Simple rules on the totals and ingredient tags.

Flag rules used in the demo

- Sugar ≥ 25 g or fast carbs → Not suitable for diabetics
- ≥ 600 kcal → High calorie meal
- Fat ≥ 25 g → High in fat
- Sodium ≥ 600 mg → Moderate sodium
- Gluten / dairy / nuts → Contains allergens
- Fiber ≥ 5 g, protein ≥ 15 g → Positive notes

Sprint 2 implementation

A cross-platform prototype so the whole team can run it on any device

Expo · React Native + TypeScript

One codebase runs on the iOS Simulator, Android, a real phone through Expo Go, and in the browser.

Photo input

expo-image-picker handles camera and gallery with the required permissions declared in app.json.

Recognition and nutrition

A single **analyseImage()** function returns the detected meal. Nutrition values live in a per-100 g table and are scaled by the confirmed grams.

Screens and state

Home, Confirm and Summary screens with a lightweight navigator. Saved meals are kept in app state for this sprint.

Sprint 2 test cases

Run manually on the web build and the iOS Simulator

Area	Test case	Expected result	Outcome
Photo input	Open the demo meal	Confirm screen opens with the photo and “Analysing…” state	**Pass**
Recognition	Wait for analysis	Masoub detected with 91% match and 6 ingredients	**Pass**
Confirmation	Tap + on Tamees bread	Grams increase and live total rises	**Pass**
Confirmation	Remove Honey	Row disappears, total drops from 746 to 699 kcal	**Pass**
Confirmation	Add Honey back	Row returns, total becomes 760 kcal	**Pass**
Nutrition	Tap Estimate nutrition	Summary shows kcal, macros, score and breakdown	**Pass**
Flags	View health flags	Diabetes, calorie, fat, sodium, allergen and positive flags shown	**Pass**
Save	Tap Save meal	Home shows the meal under Recent meal and in History	**Pass**

All documented Sprint 2 test cases passed.

Sprint 2 results

Stories 003–005 delivered, with an early start on 006

01 **Photo input**
Camera, gallery and demo image reach the confirm screen.

02 **Food recognition**
Dish, confidence and ingredients are proposed from the photo.

03 **Food confirmation**
Grams, removals and additions update the estimate live.

04 **Nutrition and flags**
Calories, macros, health score and flags, saved to history.

Sprint 2

COMPLETE

Photo, recognition and confirmation scope

Future work

Replace the mock with a real AI model and complete the remaining stories

Real AI food recognition

Connect a vision model so the prediction comes from the photo itself: the dish, its ingredients, and an estimated portion size for each one.

Nutrition database

Look up confirmed foods in a nutrition API instead of the built-in table, so any dish can be estimated.

Cloud history (007–008)

Store saved meals per account in Firebase so history follows the user across devices.

Personal flags

Let the user set conditions such as diabetes or allergies so the flags are tailored to them, with Arabic language support.

MealLens

Thank You

Questions?

Sprint 2